Table of Contents

🌐 Week 9 Hands-On Guide: Introduction to Web Development with Spring Boot

- Step-by-Step Tutorial for Students — Your First REST API
- 📋 Before You Start — Checklist
- 🗺️ The Big Picture — What Changes This Week

📘 PART A — Web Concepts You Must Know First

- A1 · Client-Server Architecture
- A2 · HTTP Protocol
  - HTTP Methods — The Four Core Verbs
  - HTTP Status Codes — What the Server Tells You Back
- A3 · REST API Principles
  - Rule 1 — Use Nouns in URLs, Not Verbs
  - Rule 2 — Use Plural Nouns for Collections
  - Rule 3 — Nest Resources for Relationships
  - Rule 4 — Use Query Parameters for Filtering
- A4 · JSON Data Format
  - JSON Syntax
  - JSON Data Types
  - Java Object ↔ JSON (Spring Boot does this automatically)
- A5 · Spring Boot Overview

💻 PART B — Set Up the Project

- Step 1 — Create a Spring Boot Module in IntelliJ IDEA
  - Option A — Using Spring Initializr Website (Recommended)
  - Option B — Create Directly Inside IntelliJ IDEA
- Step 2 — Understand the Project Structure
- Step 3 — Create the Package Structure

💻 PART C — Write the Code (File by File)

- FILE 1 of 5 — `Student.java` (Model)
- FILE 2 of 5 — `ApiResponse.java` (Model)
- FILE 3 of 5 — `HelloController.java` (Controller)
- FILE 4 of 5 — `StudentController.java` (Controller)
- FILE 5 of 5 — `GlobalExceptionHandler.java` (Controller)

Step 1 — Start the Server
Step 2 — Test in Your Web Browser
Step 3 — Install and Set Up Postman
Step 4 — Test All Endpoints in Postman
GET — Read All Students
GET — Read One Student by ID
GET — Student Not Found (404)
GET — Filter by Major
POST — Add a New Student
POST — Validation Error (400)
PUT — Update an Existing Student
DELETE — Remove a Student
🔍 PART E — Understanding How Everything Connects
Annotation Summary — Everything You Used This Week
⚠️ Common Mistakes to Avoid
Mistake 1 — Missing `@RestController` annotation
Mistake 2 — No getter methods on model class (Jackson cannot serialize)
Mistake 3 — No no-arg constructor (Jackson cannot deserialize request body)
Mistake 4 — Wrong URL path order (specific routes before generic ones)
Mistake 5 — Testing POST without Content-Type header
Mistake 6 — Port conflict
🏆 Extension Challenges (Homework)
Challenge 1 — Add a `BookController` (Required)
Challenge 2 — Query Parameter Enhancements (Required)
Challenge 3 — Calculator API (Challenge)
Challenge 4 — Custom Port and Startup Message (Challenge)
📌 Quick Reference Card
REST URL Design Pattern
HTTP Status Codes to Use
Week 9 vs Week 8 — Side by Side

🌐 Week 9 Hands-On Guide: Introduction to Web Development with Spring Boot

Step-by-Step Tutorial for Students — Your First REST API

> ***What you will build:*** *A fully working* ***Spring Boot REST API*** *that runs on your machine as a web server. By the end of this session you will have real HTTP endpoints you can call from a browser and from Postman — the first step from console apps into modern web development.*

📋 Before You Start — Checklist

Make sure you have all of these ready before writing a single line of code:

Requirement	Where to get it
✅ IntelliJ IDEA (Community or Ultimate)	https://www.jetbrains.com/idea/download/
✅ Java JDK 17 or higher	https://adoptium.net/
✅ Internet connection (for Spring Initializr)	Required first time only
✅ Postman (API testing tool)	https://www.postman.com/downloads/
✅ Web browser (Chrome, Firefox, Edge)	Already installed

> 💡 ***Coming from Week 8?*** *You do not need PostgreSQL or pgAdmin for this week. We are building an in-memory REST API — no database yet. Database integration returns in Week 11.*

🗺️ The Big Picture — What Changes This Week

```
Weeks 1–8  (Console Application)
──────────────────────────────────────────────────────────
  User types input in terminal
```

```
Java program processes it
        ↓
  Output printed to terminal
  (Only one user at a time, only on this machine)


Week 9+  (Web Application)
──────────────────────────────────────────────────────
  Browser / Mobile App / Postman
        ↓  HTTP Request (GET, POST, PUT, DELETE)
  Spring Boot Server (your Java code)
        ↓  HTTP Response (JSON)
  Browser / Mobile App / Postman
  (Many users at once, from anywhere in the world)
```

📘 PART A — Web Concepts You Must Know First

> ***Read this before touching any code.*** *These concepts are the foundation of everything you will build from Week 9 onwards.*

A1 · Client-Server Architecture

Every web application is split into two sides:

Side	Name	What it does	Examples
Frontend	Client	What the user sees and interacts with	Chrome browser, mobile app, Postman
Backend	Server	Handles logic, talks to database, sends data	Spring Boot app (your Java code)

```
CLIENT                              SERVER
(Browser / Postman / Mobile App)    (Your Spring Boot App)

     HTTP Request
     ──────────────────────────────►
     GET /api/students

                                    1. Receive the request
                                    2. Run Java business logic
                                    3. Build JSON response
```

```
HTTP Response (JSON)
     ←──────────────────────────────
     [{"id":1,"name":"Alice"},  ... ]
```

Why separate them?
- Multiple clients (web, mobile, desktop) share the same backend
- Frontend and backend teams can work independently
- Scale the server separately from the client
- Business logic stays hidden on the server (security)

A2 · HTTP Protocol

HTTP (HyperText Transfer Protocol) is the language that clients and servers use to talk to each other. Every web request follows the same structure.

HTTP Methods — The Four Core Verbs

Method	Purpose	Example URL	Has Request Body?
GET	Read / retrieve data	`GET /api/students`	No
POST	Create a new resource	`POST /api/students`	Yes
PUT	Update an existing resource	`PUT /api/students/1`	Yes
DELETE	Remove a resource	`DELETE /api/students/1`	No

HTTP Status Codes — What the Server Tells You Back

Code	Meaning	When you see it
200 OK	Request succeeded	GET returned data successfully
201 Created	New resource was created	POST added a new student
400 Bad Request	Your request had invalid data	Missing required field
404 Not Found	Resource does not exist	Student ID not found
500 Internal Server Error	Server crashed	Unhandled exception in Java code

A3 · REST API Principles

REST (REpresentational State Transfer) is the standard way to design web APIs. Follow these rules and your API will be predictable and easy to use.

Rule 1 — Use Nouns in URLs, Not Verbs

```
WRONG (verbs in URL):
  GET /api/getStudents
  POST /api/createStudent
  GET /api/deleteStudent?id=1

CORRECT (nouns + HTTP method defines the action):
  GET    /api/students          → read all students
  POST   /api/students          → create one student
  GET    /api/students/1        → read student with id=1
  PUT    /api/students/1        → update student with id=1
  DELETE /api/students/1        → delete student with id=1
```

Rule 2 — Use Plural Nouns for Collections

```
WRONG:   /api/student
CORRECT: /api/students

WRONG:   /api/course
CORRECT: /api/courses
```

Rule 3 — Nest Resources for Relationships

```
GET  /api/students/1/courses       → courses for student 1
POST /api/students/1/courses       → enroll student 1 in a course
GET  /api/departments/1/students   → all students in department 1
```

Rule 4 — Use Query Parameters for Filtering

```
GET /api/students?major=Computer+Science   → filter by major
GET /api/students?gpa_min=3.5             → students with GPA ≥ 3.5
```

A4 · JSON Data Format

JSON (JavaScript Object Notation) is the standard data format for REST APIs. Your Spring Boot app will automatically convert Java objects to JSON and back.

JSON Syntax

```
{
  "studentId": "S001",
  "name": "Alice Johnson",
  "age": 20,
  "email": "alice@university.edu",
  "gpa": 3.8,
  "isActive": true,
  "major": "Computer Science",
  "enrolledCourses": ["CS101", "MATH101"],
  "address": {
    "city": "Phnom Penh",
    "country": "Cambodia"
  }
}
```

JSON Data Types

Type	Example
String	`"Alice Johnson"`
Number	`20`, `3.8`
Boolean	`true`, `false`
Array	`["CS101", "MATH101"]`
Object	`{ "city": "Phnom Penh" }`
Null	`null`

Java Object ↔ JSON (Spring Boot does this automatically)

```
// Your Java object                    // Becomes this JSON automatically
public class Student {          →    {
    String name = "Alice";      →      "name": "Alice",
    int age = 20;               →      "age": 20,
    double gpa = 3.8;           →      "gpa": 3.8
}                               →    }
```

> 💡 ***Key rule:*** *Spring Boot uses* ***Jackson*** *(a library) to convert Java objects to JSON. It reads your* ***getter methods*** *to get field values. If a field has no getter, it will NOT appear in the JSON.*

A5 · Spring Boot Overview

Spring Boot is the most popular Java framework for building web APIs. It handles the complicated server setup so you can focus on writing your business logic.

```
Spring Ecosystem
  ├── Spring Core      → Dependency Injection (IoC container)
  ├── Spring Boot      → Quick setup, auto-configuration, embedded server
  ├── Spring MVC       → Web layer — handles HTTP requests/responses
  ├── Spring Data JPA  → Database access (Week 11)
  └── Spring Security  → Authentication and authorisation (later)
```

What Spring Boot gives you automatically:
- An embedded Tomcat web server (no separate installation)
- JSON conversion via Jackson (no extra configuration)
- Auto-configuration based on your dependencies
- A clean project structure that scales to enterprise apps

💻 PART B — Set Up the Project

Step 1 — Create a Spring Boot Module in IntelliJ IDEA

Spring Boot projects use **Maven** as the build system, so this week's module is created differently from previous weeks.

Option A — Using Spring Initializr Website (Recommended)

1. Open your browser and go to **https://start.spring.io/**
2. Fill in the project settings:

Field	Value
Project	Maven
Language	Java
Spring Boot	3.2.x (choose the latest stable — not SNAPSHOT)
Group	`com.university`
Artifact	`week9-api`
Name	`week9-api`
Description	Week 9 REST API Demo
Package name	`com.university.week9api`
Packaging	Jar
Java	17

3. Click **"ADD DEPENDENCIES"** and search for then select:
   - **Spring Web** ← this is the only dependency you need today
4. Click the **"GENERATE"** button — this downloads a `.zip` file
5. **Extract** the ZIP to your `JavaCourse/` project folder so it sits alongside your other week folders:

```
JavaCourse/
├── Week1-Basics/
├── Week2-OOP/
├── ...
├── Week7-JDBC/
├── Week8-JDBC-Project/
└── week9-api/              ← extracted here
    ├── src/
    └── pom.xml
```

6. In **IntelliJ IDEA**: **File** → **Open** → navigate to the `week9-api` folder → click **Open**

7. IntelliJ will detect it is a Maven project and ask to **"Load Maven Project"** — click it. Wait for all dependencies to download (watch the progress bar at the bottom).

Option B — Create Directly Inside IntelliJ IDEA

1. In IntelliJ, go to **File** → **New** → **Module...**
2. In the left panel, select **"Spring Initializr"**

> *If you do not see Spring Initializr, use **Option A** instead.*

3. Fill in the same settings as Option A above
4. Click **Next** → select **Spring Web** → click **Create**

Step 2 — Understand the Project Structure

After the project opens, expand the folders in the Project panel. You will see:

```
week9-api/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/university/week9api/
│   │   │       └── Week9ApiApplication.java     ← Main class - entry point
│   │   └── resources/
│   │       └── application.properties           ← Configuration file
│   └── test/
│       └── java/
│           └── com/university/week9api/
│               └── Week9ApiApplicationTests.java
├── pom.xml                                       ← Dependencies (like build.gradle)
└── mvnw                                          ← Maven wrapper script
```

What each file does:

`Week9ApiApplication.java` — This starts the entire application. You will rarely edit this file.

```java
package com.university.week9api;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```java
@SpringBootApplication   // Enables auto-configuration + component scanning
public class Week9ApiApplication {    public static void main(String[] args) {
        SpringApplication.run(Week9ApiApplication.class, args);
        // This line starts the embedded Tomcat server on port 8080    }    }
```

`application.properties` — Application settings. Add these lines:

```
# Server port (default is 8080 - change if something else is already using it)
server.port=8080

# Application name (shows in logs)
spring.application.name=week9-api

# Show more detailed startup logs
logging.level.org.springframework.web=DEBUG
```

`pom.xml` — Your dependencies. It should already contain:

```xml
<dependencies>
    <!— Spring Web: includes Tomcat server + Spring MVC + Jackson JSON —>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!— Spring Boot test utilities —>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Step 3 — Create the Package Structure

Inside `src/main/java/com/university/week9api/`, create these sub-packages:

1. Right-click `week9api` → **New** → **Package** → type `controller`
2. Right-click `week9api` → **New** → **Package** → type `model`

Your final package layout:

```
com/university/week9api/
```

```
├── controller/                 ← create this
└── model/                      ← create this
```

> 📌 ***Create all packages before creating any Java files.*** *Each Java file must be placed in the correct package.*

💻 PART C — Write the Code (File by File)

> *Write the files in the order shown. Each section builds on the one before it.*

FILE 1 of 5 — `Student.java` (Model)

What this file does: Represents a student as a Java object. Spring Boot will automatically convert this to/from JSON for every API request and response.

Create `Student.java` inside the `model` package:

```java
package com.university.week9api.model;

/**
 * Student - model class representing a student resource.
 *
 * Spring Boot (Jackson) converts this class to JSON automatically.
 * IMPORTANT: Jackson reads getter methods to build the JSON.
 *            Every field that should appear in JSON MUST have a getter.
 *
 * JSON output example:
 * {
 *   "id": 1,
 *   "name": "Alice Johnson",
 *   "age": 20,
 *   "major": "Computer Science",
 *   "gpa": 3.8
 * }
 */
public class Student {

    private int    id;
    private String name;
    private int    age;
    private String major;
    private double gpa;

    // ── No-arg constructor - required by Jackson for JSON deserialization ──
```

```
    // When a POST request arrives with JSON body, Jackson needs this
    // constructor to create a Student object before filling in the fields.
    public Student() {}
    // ── All-args constructor — used when we create sample data ──────────────
    public Student(int id, String name, int age, String major, double gpa) {
        this.id    = id;        this.name  = name;        this.age   = age;
        this.major = major;        this.gpa   = gpa;    }
    // ── Getters — Jackson reads these to produce JSON ────────────────────────
    public int    getId()    { return id; }    public String getName()  { return name; }
    public int    getAge()   { return age; }    public String getMajor() { return major; }
    public double getGpa()   { return gpa; }
    // ── Setters — Jackson uses these when reading JSON from a POST body ───────
    public void setId(int id)       { this.id = id; }
    public void setName(String name){ this.name = name; }
    public void setAge(int age)     { this.age = age; }
    public void setMajor(String major){ this.major = major; }
    public void setGpa(double gpa)  { this.gpa = gpa; }            @Override
    public String toString() {
        return String.format("Student[id=%d, name=%s, age=%d, major=%s, gpa=%.2f]",
            id, name, age, major, gpa);    }    }
```

FILE 2 of 5 — `ApiResponse.java` (Model)

What this file does: A reusable wrapper for consistent API responses. Instead of returning raw data or raw error messages, every endpoint returns a structured response.

Create `ApiResponse.java` inside the `model` package:

```
package com.university.week9api.model;

/**
 * ApiResponse — standard response wrapper for all API endpoints.
 *
 * Using a consistent response format makes your API predictable.
 * Every response (success OR error) has the same structure:
 *
 * Success example:
 * {
 *   "success": true,
 *   "message": "Student found",
 *   "data": { "id": 1, "name": "Alice" }
 * }
 *
 * Error example:
 * {
 *   "success": false,
 *   "message": "Student not found with ID: 99",
```

```
 *   "data": null * }      */      public class ApiResponse {
    private boolean success;    private String  message;
    private Object  data;     // Can hold any Java object - Student, List, String, etc.
            // ── Constructors ──────────────────────────────────────────────────────
    public ApiResponse(boolean success, String message, Object data) {
        this.success = success;        this.message = message;        this.data    = data;
    }
    // ── Static factory methods - convenient to use in controllers ────────────
    /** Call this when the operation succeeded and you have data to return. */
    public static ApiResponse ok(String message, Object data) {
        return new ApiResponse(true, message, data);    }
    /** Call this when the operation succeeded but there is no data to return. */
    public static ApiResponse ok(String message) {
        return new ApiResponse(true, message, null);    }
    /** Call this when something went wrong. */
    public static ApiResponse error(String message) {
        return new ApiResponse(false, message, null);    }
    // ── Getters - required for Jackson JSON serialization ────────────────────
    public boolean isSuccess() { return success; }
    public String  getMessage(){ return message; }
    public Object  getData()   { return data; }}
```

FILE 3 of 5 — `HelloController.java` (Controller)

What this file does: Your very first REST controller. It handles simple HTTP GET requests and shows you how all the core annotations work.

Create `HelloController.java` inside the `controller` package:

```
package com.university.week9api.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.HashMap;
import java.util.Map;

/**
 * HelloController - demonstrates the four most important Spring MVC annotations.
 *
 * Base URL for all endpoints in this controller: /api/hello
 *
```

```
 * Annotation summary:
 *   @RestController    = this class handles HTTP requests and returns data (not a web page)
 *   @RequestMapping    = sets the base URL path for all methods in this class
 *   @GetMapping        = handles HTTP GET requests
 *   @PathVariable      = reads a value from inside the URL  e.g. /hello/{name}
 *   @RequestParam      = reads a value from the query string e.g. /hello?name=Alice */
@RestController@RequestMapping("/api/hello")public class HelloController {
    // ── GET /api/hello ──────────────────────────────────────────────────────
    // Simplest possible endpoint - returns a plain String    @GetMapping
    public String sayHello() {        return "Hello from Spring Boot! Week 9 API is running.";
    }
    // ── GET /api/hello/{name} ───────────────────────────────────────────────
    // @PathVariable reads {name} directly from the URL path
    // Example: GET /api/hello/Alice  →  name = "Alice"    @GetMapping("/{name}")
    public String sayHelloToName(@PathVariable String name) {
        return "Hello, " + name + "! Welcome to the Spring Boot API.";    }
    // ── GET /api/hello/greet?name=Alice ─────────────────────────────────────
    // @RequestParam reads the query string parameter
    // Example: GET /api/hello/greet?name=Alice  →  name = "Alice"
    // required = false means the parameter is optional; defaultValue is used if missing
    @GetMapping("/greet")    public String greetWithParam(
            @RequestParam(required = false, defaultValue = "World") String name) {
        return "Greetings, " + name + "!";    }
    // ── GET /api/hello/info ─────────────────────────────────────────────────
    // Returns a JSON object (Map becomes a JSON object automatically)    @GetMapping("/info")
    public Map<String, String> getAppInfo() {
        Map<String, String> info = new HashMap<>();
        info.put("appName",    "Week 9 REST API");        info.put("version",    "1.0.0");
        info.put("framework",  "Spring Boot 3.2");
        info.put("developer",  "KhmerSide Java Course");
        info.put("serverTime", LocalDateTime.now()
                    .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")));
        return info;    }
    // ── GET /api/hello/add?a=5&b=3 ──────────────────────────────────────────
    // Multiple query parameters in one endpoint
    // Example: GET /api/hello/add?a=10&b=4  →  returns 14    @GetMapping("/add")
    public Map<String, Object> addNumbers(            @RequestParam int a,
            @RequestParam int b) {        Map<String, Object> result = new HashMap<>();
        result.put("a",      a);        result.put("b",      b);
        result.put("sum",    a + b);
        result.put("operation", a + " + " + b + " = " + (a + b));        return result;
    }    }
```

FILE 4 of 5 — `StudentController.java` (Controller)

What this file does: A complete REST controller for Student CRUD operations using an in-memory list. This mirrors the `StudentManagementApp` from Week 8, but over HTTP instead of the console.

Create `StudentController.java` inside the `controller` package:

```java
package com.university.week9api.controller;

import com.university.week9api.model.ApiResponse;
import com.university.week9api.model.Student;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

/**
 * StudentController — full CRUD REST API for students.
 *
 * Base URL: /api/students
 *
 * Endpoints:
 *   GET    /api/students              → all students
 *   GET    /api/students/{id}         → one student by ID
 *   GET    /api/students?major=...    → filter by major
 *   GET    /api/students/count        → total count
 *   POST   /api/students              → add new student
 *   PUT    /api/students/{id}         → update entire student
 *   DELETE /api/students/{id}         → delete student
 *
 * Data storage: in-memory ArrayList (no database this week).
 * NOTE: data resets every time the server restarts.
 * Database integration comes in Week 11.
 *
 * New annotation this week:
 *   @RequestBody   = reads the JSON body of a POST/PUT request
 *                    and converts it into a Java object automatically
 *   ResponseEntity = lets you set the HTTP status code on the response
 *                    (e.g. 201 Created instead of default 200 OK)
 */
@RestController
@RequestMapping("/api/students")
public class StudentController {
```

```java
        // ── In-memory data store ──────────────────────────────────────────
    // This replaces the database for now.
    // Using the same data as university_db Week 6 sample data.
    private final List<Student> students = new ArrayList<>();
    private int nextId = 9;   // next ID to assign when a new student is added
    // ── Constructor: load sample data matching Week 6 university_db ──────
    public StudentController() {
        students.add(new Student(1, "Alice Johnson", 20, "Computer Science", 3.8));
        students.add(new Student(2, "Bob Smith",     22, "Mathematics",      3.5));
        students.add(new Student(3, "Charlie Brown", 21, "Physics",          3.9));
        students.add(new Student(4, "Diana Prince",  20, "Computer Science", 3.7));
        students.add(new Student(5, "Eve Wilson",    23, "Chemistry",        3.6));
        students.add(new Student(6, "Frank Castle",  24, "Engineering",      3.4));
        students.add(new Student(7, "Grace Hopper",  19, "Computer Science", 4.0));
        students.add(new Student(8, "Henry Ford",    21, "Engineering",      3.3));    }
        // ═══════════════════════════════════════════════════════════════════
    //  GET — Read operations
    // ═══════════════════════════════════════════════════════════════════
    /**      * GET /api/students     * GET /api/students?major=Computer+Science     *
     * Returns all students, or filters by major if the query param is provided.
     * ResponseEntity lets us control the HTTP status code (200 OK here).     */
    @GetMapping    public ResponseEntity<ApiResponse> getAllStudents(
            @RequestParam(required = false) String major) {
        List<Student> result;                if (major ≠ null && !major.isEmpty()) {
            // Filter by major — case-insensitive partial match
            result = students.stream()
                    .filter(s → s.getMajor().toLowerCase()
                                 .contains(major.toLowerCase()))
                    .collect(Collectors.toList());                    if (result.isEmpty()) {
                return ResponseEntity.status(HttpStatus.NOT_FOUND)
                        .body(ApiResponse.error("No students found in major: " + major));
            }            return ResponseEntity.ok(
                    ApiResponse.ok("Found " + result.size() + " student(s) in: " + major,
        result));
        }                 // No filter — return everyone        return ResponseEntity.ok(
                ApiResponse.ok("Found " + students.size() + " student(s)", students));
    }                /**      * GET /api/students/count     *
     * Returns the total number of students.
     * NOTE: This mapping MUST be placed BEFORE /{id} in the code,
     * otherwise Spring will try to parse "count" as an integer ID.     */
    @GetMapping("/count")    public ResponseEntity<ApiResponse> getStudentCount() {
        return ResponseEntity.ok(
                ApiResponse.ok("Total students: " + students.size(), students.size()));
    }                /**      * GET /api/students/{id}     *
     * Returns one student by their integer ID.
     * Returns 404 Not Found if no student has that ID.     */     @GetMapping("/{id}")
    public ResponseEntity<ApiResponse> getStudentById(@PathVariable int id) {
        Optional<Student> found = students.stream()
                .filter(s → s.getId() == id)                .findFirst();
        if (found.isPresent()) {            return ResponseEntity.ok(
```

```
                ApiResponse.ok("Student found", found.get()));        } else {
        // 404 Not Found - the student does not exist
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(ApiResponse.error("No student found with ID: " + id));        }
}
// ==================================================================
//  POST - Create operation
// ==================================================================
/**      * POST /api/students
 * Body: { "name": "New Student", "age": 21, "major": "Physics", "gpa": 3.5 }     *
 * @RequestBody reads the JSON request body and converts it to a Student object.
 * Returns 201 Created (not the default 200 OK) to follow REST conventions.     */
@PostMapping
public ResponseEntity<ApiResponse> addStudent(@RequestBody Student student) {
    // Validate required fields
    if (student.getName() == null || student.getName().isEmpty()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(ApiResponse.error("Student name is required."));        }
    if (student.getMajor() == null || student.getMajor().isEmpty()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(ApiResponse.error("Student major is required."));        }
    if (student.getGpa() < 0.0 || student.getGpa() > 4.0) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(ApiResponse.error("GPA must be between 0.0 and 4.0."));        }
            // Assign the next ID automatically        student.setId(nextId++);
    students.add(student);
    // Return 201 Created with the newly added student
    return ResponseEntity.status(HttpStatus.CREATED)
            .body(ApiResponse.ok("Student added successfully!", student));    }
        // ==================================================================
//  PUT - Update operation
// ==================================================================
/**      * PUT /api/students/{id}
 * Body: { "name": "Updated Name", "age": 22, "major": "Engineering", "gpa": 3.6 }
 *       * Updates ALL fields of the student with the given ID.
 * Returns 404 if the student is not found.     */     @PutMapping("/{id}")
public ResponseEntity<ApiResponse> updateStudent(            @PathVariable int id,
        @RequestBody Student updatedStudent) {
    for (int i = 0; i < students.size(); i++) {
        if (students.get(i).getId() == id) {
            // Preserve the original ID - do not let the client change it
            updatedStudent.setId(id);                students.set(i, updatedStudent);
                    return ResponseEntity.ok(
                    ApiResponse.ok("Student updated successfully!", updatedStudent));
        }        }                return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ApiResponse.error("No student found with ID: " + id));    }
// ==================================================================
//  DELETE - Remove operation
// ==================================================================
/**      * DELETE /api/students/{id}     *
```

```
     * Removes the student with the given ID from the list.
     * Returns 404 if the student is not found.     */     @DeleteMapping("/{id}")
    public ResponseEntity<ApiResponse> deleteStudent(@PathVariable int id) {
        Optional<Student> found = students.stream()
                .filter(s -> s.getId() == id)                .findFirst();
        if (found.isPresent()) {            students.remove(found.get());
            return ResponseEntity.ok(
                    ApiResponse.ok("Student with ID " + id + " deleted successfully."));
        }                 return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(ApiResponse.error("No student found with ID: " + id));    }    }
```

FILE 5 of 5 — `GlobalExceptionHandler.java` (Controller)

What this file does: Catches unhandled exceptions anywhere in the application and returns a clean JSON error instead of the ugly default Spring error page.

Create `GlobalExceptionHandler.java` inside the `controller` package:

```java
package com.university.week9api.controller;

import com.university.week9api.model.ApiResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

/**
 * GlobalExceptionHandler — catches unhandled exceptions across ALL controllers.
 *
 * Without this, Spring Boot returns a generic white-label HTML error page
 * when something goes wrong. This handler intercepts exceptions and returns
 * clean JSON responses instead, making the API easier to use.
 *
 * @RestControllerAdvice  = applies to ALL @RestController classes in the project
 * @ExceptionHandler      = marks a method as the handler for a specific exception type
 */
@RestControllerAdvice
public class GlobalExceptionHandler {

    /**
     * Handles the case where a path variable or query param has the wrong type.
     * Example: GET /api/students/abc  (where abc should be an int)
     */
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiResponse> handleTypeMismatch(
            MethodArgumentTypeMismatchException ex) {
```

```
                String message = "Invalid parameter '" + ex.getName() + "': " +
                        "expected a " + ex.getRequiredType().getSimpleName() +
                        " but received '" + ex.getValue() + "'";
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(ApiResponse.error(message));    }                /**
     * Catches any other exception not handled elsewhere.
     * Returns 500 Internal Server Error.     */     @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse> handleGenericException(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(ApiResponse.error("Server error: " + ex.getMessage()));    }    }
```

🏃 PART D — Run and Test the Application

Step 1 — Start the Server

1. In the **Project** panel, find and open `Week9ApiApplication.java`
2. Right-click anywhere inside the file → **Run 'Week9ApiApplication'**
3. Watch the **Run** panel at the bottom — wait for this line to appear:

```
Tomcat started on port(s): 8080 (http)
Started Week9ApiApplication in 3.2 seconds
```

> ⚠️ ***Port already in use?*** *If you see* `Port 8080 was already in use`*, open* `application.properties` *and change* `server.port=8080` *to* `server.port=8081`*, then restart.*

Step 2 — Test in Your Web Browser

Open Chrome or Firefox and visit these URLs one by one:

URL	Expected Response
`http://localhost:8080/api/hello`	`Hello from Spring Boot! Week 9 API is running.`
`http://localhost:8080/api/hello/Alice`	`Hello, Alice! Welcome to the Spring Boot API.`

`name=Bob`	`Greetings, Bob!`
`http://localhost:8080/api/hello/greet`	`Greetings, World!` (default value)
`http://localhost:8080/api/hello/add?a=5` (then add `b=3`)	JSON: `{ "sum": 8, ... }`
`http://localhost:8080/api/hello/info`	JSON object with app info
`http://localhost:8080/api/students`	JSON array of 8 students
`http://localhost:8080/api/students/1`	JSON: Alice Johnson
`http://localhost:8080/api/students/99`	JSON: `{ "success": false, "message": "No student found ..." }`
`http://localhost:8080/api/students?major=Computer+Science`	JSON: Alice, Diana, Grace
`http://localhost:8080/api/students/count`	JSON: `{ "data": 8 }`

> 💡 ***Browser tip:*** *Browsers can only do **GET** requests from the address bar. To test POST, PUT, and DELETE you must use Postman.*

Step 3 — Install and Set Up Postman

1. Go to **https://www.postman.com/downloads/** and download Postman
2. Install and open it
3. Click **"New"** → **"Collection"** → name it `Week 9 – Spring Boot API`
4. Save all requests below into this collection so you can reuse them

---

Step 4 — Test All Endpoints in Postman

Work through every test below in order. Each one tests a different HTTP method.

GET — Read All Students

```
Method : GET
URL    : http://localhost:8080/api/students
Headers: (none needed)
Body   : (none)
```

Expected Response (200 OK):

```
{
  "success": true,
  "message": "Found 8 student(s)",
  "data": [
    { "id": 1, "name": "Alice Johnson", "age": 20, "major": "Computer Science", "gpa": 3.8 },
    { "id": 2, "name": "Bob Smith",     "age": 22, "major": "Mathematics",      "gpa": 3.5 },
    ...
  ]
}
```

GET — Read One Student by ID

```
Method : GET
URL    : http://localhost:8080/api/students/7
```

Expected Response (200 OK):

```
{
  "success": true,
  "message": "Student found",
  "data": { "id": 7, "name": "Grace Hopper", "age": 19, "major": "Computer Science", "gpa":
        4.0 }
}
```

GET — Student Not Found (404)

```
Method : GET
URL    : http://localhost:8080/api/students/999
```

Expected Response (404 Not Found):

```
{
  "success": false,
  "message": "No student found with ID: 999",
  "data": null
}
```

GET — Filter by Major

```
Method : GET
URL    : http://localhost:8080/api/students?major=Computer+Science
```

Expected Response (200 OK):

```
{
  "success": true,
  "message": "Found 3 student(s) in: Computer Science",
  "data": [
    { "id": 1, "name": "Alice Johnson", ... },
    { "id": 4, "name": "Diana Prince",  ... },
    { "id": 7, "name": "Grace Hopper",  ... }
  ]
}
```

POST — Add a New Student

```
Method  : POST
URL     : http://localhost:8080/api/students
Headers : Content-Type: application/json
Body    : raw → JSON
```

Request Body:

```
{
  "name": "Sokha Chan",
  "age": 22,
  "major": "Computer Science",
```

```
  "gpa": 3.75}
```

Expected Response (201 Created):

```
{
  "success": true,
  "message": "Student added successfully!",
  "data": { "id": 9, "name": "Sokha Chan", "age": 22, "major": "Computer Science", "gpa": 3.75
        }
}
```

> 📌 ***How to set the Content-Type header in Postman:***
> *1. Click the "**Body**" tab*
> *2. Select "**raw**"*
> *3. From the dropdown on the right, choose "**JSON**"*
> *4. Postman automatically adds* `Content-Type: application/json`

POST — Validation Error (400)

```
Method  : POST
URL     : http://localhost:8080/api/students
Headers : Content-Type: application/json
Body    :
```

```
{
  "name": "",
  "age": 22,
  "major": "Physics",
  "gpa": 3.5
}
```

Expected Response (400 Bad Request):

```
{
  "success": false,
  "message": "Student name is required.",
  "data": null
}
```

PUT — Update an Existing Student

```
Method  : PUT
URL     : http://localhost:8080/api/students/9
Headers : Content-Type: application/json
Body    : raw → JSON
```

Request Body:

```
{
  "name": "Sokha Chan",
  "age": 23,
  "major": "Software Engineering",
  "gpa": 3.9
}
```

Expected Response (200 OK):

```
{
  "success": true,
  "message": "Student updated successfully!",
  "data": { "id": 9, "name": "Sokha Chan", "age": 23, "major": "Software Engineering", "gpa":
        3.9 }
}
```

DELETE — Remove a Student

```
Method : DELETE
URL    : http://localhost:8080/api/students/9
```

Expected Response (200 OK):

```
{
  "success": true,
  "message": "Student with ID 9 deleted successfully.",
  "data": null
}
```

Verify the deletion:

```
Method : GET
URL    : http://localhost:8080/api/students/9
```

Expected Response (404 Not Found):

```
{
  "success": false,
  "message": "No student found with ID: 9",
  "data": null
}
```

🔍 PART E — Understanding How Everything Connects

```
Postman sends: POST /api/students
               Content-Type: application/json
               Body: { "name": "Sokha Chan", "age": 22, ... }
       │
       ▼
Embedded Tomcat (inside Spring Boot) receives the HTTP request
       │
       ▼
Spring MVC router looks at the URL + HTTP method:
  → URL starts with /api/students  → StudentController
  → HTTP method is POST            → addStudent() method
       │
       ▼
Jackson reads the JSON body and creates a Student Java object:
  { "name": "Sokha Chan" }  →  new Student() with name = "Sokha Chan"
       │
       ▼
addStudent(Student student) runs:
  - Validates name, major, gpa
  - Assigns id = nextId++
  - Adds student to the ArrayList
  - Builds ApiResponse(success=true, ...)
       │
       ▼
Jackson converts ApiResponse back to JSON:
  ApiResponse object  →  { "success": true, "message": "...", "data": {...} }
       │
       ▼
Spring Boot sends HTTP Response:
  Status: 201 Created
```

```
Body: { "success": true, "message": "Student added successfully!", ... }
       │
       ▼
Postman displays the response to you
```

Annotation Summary — Everything You Used This Week

Annotation	Where	What it does
`@SpringBootApplication`	Main class	Enables auto-config, component scan, starts the app
`@RestController`	Controller class	Marks class as REST controller — returns data, not HTML
`@RequestMapping("/path")`	Controller class	Sets the base URL for all methods in this controller
`@GetMapping`	Method	Handles HTTP GET requests
`@PostMapping`	Method	Handles HTTP POST requests
`@PutMapping`	Method	Handles HTTP PUT requests
`@DeleteMapping`	Method	Handles HTTP DELETE requests
`@PathVariable`	Method param	Reads a value from `{placeholder}` in the URL path
`@RequestParam`	Method param	Reads a value from the query string (`?key=value`)
`@RequestBody`	Method param	Reads the JSON request body and converts to Java object
`@RestControllerAdvice`	Handler class	Applies `@ExceptionHandler` methods to all controllers

⚠ Common Mistakes to Avoid

Mistake 1 — Missing `@RestController` annotation

```
// WRONG – Spring does not register this as an endpoint handler
public class StudentController {
    @GetMapping("/api/students")
    public List<Student> getAll() { ... }
}

// CORRECT
@RestController
public class StudentController {
    @GetMapping("/api/students")
    public List<Student> getAll() { ... }
}
```

Mistake 2 — No getter methods on model class (Jackson cannot serialize)

```
// WRONG – no getters = Jackson returns empty JSON {}
public class Student {
    private String name;   // Jackson cannot read this!
}

// CORRECT – Jackson needs getters to build JSON
public class Student {
    private String name;
    public String getName() { return name; }   // Jackson reads this
}
```

Mistake 3 — No no-arg constructor (Jackson cannot deserialize request body)

```
// WRONG – POST body cannot be converted to Student object
public class Student {
    public Student(int id, String name) { ... }   // only parameterised constructor
}

// CORRECT – Jackson needs the no-arg constructor for @RequestBody
public class Student {
    public Student() {}   // required by Jackson
    public Student(int id, String name) { ... }
}
```

Mistake 4 — Wrong URL path order (specific routes before generic ones)

```
// WRONG — Spring tries to match "count" as an int ID → exception!
@GetMapping("/{id}")      // registered first — catches /students/count
public ... getById(@PathVariable int id) { ... }

@GetMapping("/count")     // never reached for /students/count
public ... getCount() { ... }

// CORRECT — specific string routes BEFORE wildcard {id} routes
@GetMapping("/count")     // registered first — matches /students/count exactly
public ... getCount() { ... }

@GetMapping("/{id}")      // registered second — only reached for numeric IDs
public ... getById(@PathVariable int id) { ... }
```

Mistake 5 — Testing POST without Content-Type header

```
// WRONG — Spring cannot parse the body, returns 415 Unsupported Media Type
POST /api/students
Body: {"name":"Alice"}         (no Content-Type header)

// CORRECT
POST /api/students
Headers: Content-Type: application/json
Body: {"name":"Alice"}
```

Mistake 6 — Port conflict

```
// Error message:
Web server failed to start. Port 8080 was already in use.

// Fix — change the port in application.properties:
server.port=8081

// Then test at the new port:
http://localhost:8081/api/students
```

🏆 Extension Challenges (Homework)

Challenge 1 — Add a `BookController` (Required)

Create `Book.java` model and `BookController.java` with:

```
// Book model fields:
// int id, String title, String author, String isbn, int year, boolean available

// Endpoints to implement:
// GET    /api/books              → all books
// GET    /api/books/{id}         → one book
// GET    /api/books?author=...   → filter by author
// POST   /api/books              → add new book
// PUT    /api/books/{id}         → update book
// DELETE /api/books/{id}         → delete book
// PUT    /api/books/{id}/borrow  → mark as borrowed (available = false)
// PUT    /api/books/{id}/return  → mark as returned (available = true)
```

Challenge 2 — Query Parameter Enhancements (Required)

Add these query parameters to `GET /api/students`:

```
GET /api/students?gpa_min=3.5              → students with GPA ≥ 3.5
GET /api/students?gpa_min=3.0&gpa_max=3.7 → students in GPA range
GET /api/students?sort=gpa&order=desc      → sort by GPA descending
GET /api/students?major=CS&gpa_min=3.5     → combined filters
```

Hint: use Java Streams `.filter()` and `.sorted()`.

Challenge 3 — Calculator API (Challenge)

Create a `CalculatorController` with these endpoints:

```
GET /api/calc/add?a=5&b=3          → { "result": 8,  "expression": "5 + 3 = 8"  }
GET /api/calc/subtract?a=10&b=4    → { "result": 6,  "expression": "10 - 4 = 6" }
GET /api/calc/multiply?a=3&b=7     → { "result": 21, "expression": "3 * 7 = 21" }
GET /api/calc/divide?a=20&b=4      → { "result": 5,  "expression": "20 / 4 = 5" }
GET /api/calc/divide?a=10&b=0      → 400 error: "Cannot divide by zero"
GET /api/calc/power?base=2&exp=10  → { "result": 1024 }
```

Challenge 4 — Custom Port and Startup Message (Challenge)

In `Week9ApiApplication.java`, add a startup log after the server starts:

```java
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class Week9ApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(Week9ApiApplication.class, args);
    }

    @Bean
    public CommandLineRunner startupMessage() {
        return args -> {
            System.out.println("╔═══════════════════════════════════════╗");
            System.out.println("║     Week 9 REST API is now running!     ");
            System.out.println("║  http://localhost:8080/api/students     ");
            System.out.println("║  http://localhost:8080/api/hello        ");
            System.out.println("╚═══════════════════════════════════════╝");
        };
    }
}
```

📌 Quick Reference Card

REST URL Design Pattern

Action	Method	URL pattern	Example
Read all	GET	`/api/resource`	`GET /api/students`
Read one	GET	`/api/resource/{id}`	`GET /api/students/1`
Filter	GET	`/api/resource?key=value`	`GET /api/students?major=CS`

Update all fields	PUT	`/api/resource/{id}`	`PUT /api/students/1`
Delete	DELETE	`/api/resource/{id}`	`DELETE /api/students/1`

HTTP Status Codes to Use

Status	Code	When to return it
OK	200	Successful GET, PUT, DELETE
Created	201	Successful POST (new resource created)
Bad Request	400	Invalid input / validation failed
Not Found	404	Resource with that ID does not exist
Server Error	500	Unhandled exception (avoid this!)

Week 9 vs Week 8 — Side by Side

Aspect	Week 8 (JDBC Console)	Week 9 (Spring Boot REST)
Interface	Terminal menu	HTTP endpoints
Input	`scanner.nextLine()`	`@RequestBody` JSON
Output	`System.out.println()`	JSON response
Communication	Local only	Network (browser, Postman, mobile)
Data storage	PostgreSQL database	In-memory ArrayList (database in Week 11)
Protocol	None (direct method calls)	HTTP (GET, POST, PUT, DELETE)
Users	One at a time	Many simultaneously

Week 9 Hands-On Guide — Introduction to Web Development with Spring Boot
From console applications to REST APIs — the foundation for modern Java web development
Next: Week 10 — Spring Boot Deep Dive (Request/Response handling, error management, CORS)